

30 Pièges Power BI qui éliminent en entretien

Le symptôme, la réponse faible du junior, la réponse attendue

En entretien Power BI, on ne te recale pas avec des fonctions exotiques. On te pose une question simple, et on regarde si tu confonds une mesure et une colonne calculée, si tu sais pourquoi un total ne retombe pas juste, ou pourquoi ta time intelligence renvoie du vide. À chaque fois, la réponse faible et la réponse attendue tiennent en deux phrases, mais ce sont ces deux phrases qui décident qui passe. Ce guide réunit les 30 pièges qui reviennent le plus, classés par thème. Pour chacun : le symptôme exact, ce que répond un junior, et ce qu'on attend de toi. Lis-le comme si tu passais l'entretien demain.

■ ■ ■ Modélisation : le socle qui décide de tout

1. Quelle est la différence entre une mesure et une colonne calculée ?

Réponse faible du junior : « C'est pareil, les deux écrivent du DAX et calculent une valeur », et il crée des colonnes calculées partout, même pour des agrégations.

Réponse attendue : Une colonne calculée est évaluée ligne par ligne dans un contexte de ligne, à l'actualisation du modèle, puis stockée et compressée dans VertiPaq : elle occupe de la mémoire et alourdit le fichier. Une mesure est évaluée à la volée, dans le contexte de filtre du visuel, et ne stocke rien. La règle : si tu veux agréger (somme, ratio, comptage) selon les filtres, c'est une mesure. Si tu veux un attribut par ligne pour filtrer, trancher dans un slicer, grouper ou créer une relation, c'est une colonne, idéalement créée en amont (à la source ou dans Power Query) plutôt qu'en DAX.

2. Pourquoi ne pas tout mettre dans une seule grande table large ?

Réponse faible du junior : « Une seule table large, c'est plus simple, j'évite les jointures et les relations. »

Réponse attendue : VertiPaq et DAX sont conçus pour le schéma en étoile : une table de faits centrale entourée de dimensions. Une table unique large répète les libellés de dimension sur chaque ligne, augmente la cardinalité, dégrade la compression, complique la time intelligence et rend les filtres croisés ambigus. L'étoile donne des chemins de filtre clairs (dimension vers faits), des mesures lisibles et un modèle nettement plus léger.

3. À quelle granularité charges-tu ta table de faits ?

Réponse faible du junior : « Je charge le détail le plus fin possible, on ne sait jamais ce qu'on voudra analyser. »

Réponse attendue : La granularité doit suivre le besoin d'analyse, pas le maximum disponible. Un grain trop fin multiplie les lignes et fait exploser la cardinalité des colonnes (clés techniques, horodatages à la seconde), ce qui plombe la compression et les temps de réponse. Tu fixes le grain métier, tu retires les colonnes à très haute cardinalité inutiles, et tu pré-agrèges en amont quand le détail n'apporte rien de plus à l'analyse.

4. Pourquoi la time intelligence a-t-elle besoin d'une table de dates dédiée?

Réponse faible du junior : «J'utilise directement la colonne date de ma table de ventes.»

Réponse attendue : Il te faut une table de dates dédiée, continue (aucune date manquante sur toute la plage couverte), reliée aux faits et marquée comme table de dates (option « Marquer comme table de dates »). Sans elle, les fonctions de time intelligence (TOTALYTD, SAMEPERIODLASTYEAR, DATEADD) renvoient des résultats faux ou vides, les axes temporels ont des trous, et tu ne peux pas comparer plusieurs tables de faits sur un axe de temps commun.

5. Tu as deux dates dans tes faits (commande et expédition). Comment les analyses-tu?

Réponse faible du junior : Il crée deux relations vers la table de dates, puis s'étonne que la seconde « ne filtre rien ».

Réponse attendue : Entre deux tables, une seule relation peut être active ; les autres existent mais restent inactives (en pointillés), et la relation active filtre par défaut. Pour calculer sur la date d'expédition, tu actives la relation inactive dans la mesure avec USERELATIONSHIP, sans dupliquer la table de dates :

```
CALCULATE(  
    [Montant],  
    USERELATIONSHIP(Faits[DateExpédition], Dates[Date])  
)
```

C'est le motif des dimensions à rôles multiples (role-playing).

6. Faut-il activer le filtrage bidirectionnel sur les relations?

Réponse faible du junior : «Je mets toutes mes relations en bidirectionnel, comme ça tout filtre tout.»

Réponse attendue : Le bidirectionnel propage les filtres dans les deux sens. Par défaut, garde le filtrage simple (dimension vers faits). Le bidirectionnel généralisé crée des chemins de filtre ambigus, dégrade les performances, peut fausser des totaux et casse souvent la sécurité au niveau des lignes (RLS). Quand tu en as réellement besoin, active-le ponctuellement sur une relation précise, ou mieux : pilote-le dans une mesure avec CROSSFILTER plutôt que de le figer sur la relation.

7. Comment modélises-tu une relation plusieurs-à-plusieurs?

Réponse faible du junior : « Je crée une relation many-to-many directe entre les deux tables, ça marche. »

Réponse attendue : Une cardinalité plusieurs-à-plusieurs directe introduit de l'ambiguïté, des totaux qui ne s'additionnent pas et des comportements de filtre contre-intuitifs. La modélisation robuste passe par une table de pont (bridge) à la bonne granularité, qui relie les deux dimensions, en maîtrisant le sens de propagation des filtres. Et tu vérifies systématiquement que les totaux restent cohérents.

■ ■ ■ DAX & mesures : écrire juste, pas seulement faire tourner

8. Comment calcules-tu un ratio comme le panier moyen ?

Réponse faible du junior : « Je divise avec l'opérateur « / », par exemple [CA] / [Nb commandes]. »

Réponse attendue : La division directe produit une erreur ou une valeur infinie quand le dénominateur vaut zéro ou est vide (BLANK). Tu utilises `DIVIDE([CA], [Nb commandes])`, qui gère ce cas proprement et renvoie BLANK par défaut, ou la valeur de repli que tu choisis en troisième argument. C'est aussi plus lisible : ça signale ton intention de gérer le zéro.

9. Quand préfères-tu SWITCH à des IF imbriqués ?

Réponse faible du junior : « J'empile des IF dans des IF, ça finit par marcher. »

Réponse attendue : Des IF imbriqués deviennent vite illisibles et difficiles à maintenir. SWITCH gère proprement plusieurs cas, et pour une suite de conditions le motif `SWITCH(TRUE(),` remplace une cascade de IF :

```
SWITCH(
    TRUE(),
    [Marge] < 0, "Perte",
    [Marge] = 0, "Seuil",
    "Profit"
)
```

C'est plus lisible, plus facile à étendre, et l'intention reste explicite.

10. À quoi servent les variables VAR dans une mesure ?

Réponse faible du junior : « Je n'en mets pas, je réécris le même calcul plusieurs fois dans la formule. »

Réponse attendue : Une variable VAR évalue une expression une seule fois et la réutilise, ce qui améliore la lisibilité et souvent la performance (le moteur ne recalcule pas la même sous-expression). Le point clé d'entretien : une VAR capture sa valeur au moment et dans le contexte où elle est définie, elle ne se réévalue pas plus loin dans la mesure.

```
VAR Ventes = [CA]
VAR Objectif = [Cible]
RETURN DIVIDE(Ventes - Objectif, Objectif)
```

11. SUMX, c'est juste une somme plus longue à écrire?

Réponse faible du junior : « SUMX et SUM, c'est pareil, j'utilise SUMX par habitude. »

Réponse attendue : SUM agrège une seule colonne. SUMX est un itérateur : il parcourt une table ligne par ligne, évalue une expression dans le contexte de ligne, puis agrège. Tu en as besoin quand le calcul se fait au niveau ligne avant la somme, par exemple SUMX(Lignes, Lignes[Qte] * Lignes[PrixUnitaire]). Mais un itérateur sur une grosse table coûte cher : si une simple SUM de colonne suffit, prends SUM. Tu n'itères que lorsque le calcul l'exige vraiment.

12. Comment comptes-tu des clients uniques?

Réponse faible du junior : « Je fais COUNT sur la colonne client. »

Réponse attendue : COUNT compte les valeurs non vides d'une colonne, pas les valeurs distinctes : tu surcomptes les clients qui reviennent. Pour des clients uniques, c'est DISTINCTCOUNT(Faits[ClientID]), ou de façon équivalente COUNTROWS(DISTINCT(Faits[ClientID])). Souviens-toi aussi que COUNTROWS compte des lignes de table, tandis que COUNT et COUNTA comptent des valeurs de colonne : ce ne sont pas les mêmes objets.

13. RELATED ou RELATEDTABLE, comment choisis-tu?

Réponse faible du junior : « Je prends celui des deux qui ne renvoie pas d'erreur. »

Réponse attendue : Le sens dépend de la cardinalité de la relation. Depuis le côté « plusieurs » (la table de faits), RELATED va chercher la valeur unique du côté « un » (la dimension) : par exemple, depuis une ligne de vente, RELATED(Produit[Categorie]). Depuis le côté « un », RELATEDTABLE renvoie la table des lignes liées du côté « plusieurs », à consommer dans un itérateur, par exemple COUNTROWS(RELATEDTABLE(Ventes)). Inverser les deux est une erreur classique.

14. Comment récupères-tu la valeur sélectionnée d'un slicer dans une mesure?

Réponse faible du junior : Il met la colonne directement dans la mesure, ce qui force une agrégation implicite ou renvoie une erreur.

Réponse attendue : Une mesure n'a pas accès à « la » valeur d'une colonne s'il y en a plusieurs dans le contexte. Tu utilises SELECTEDVALUE(Param[Devise], "EUR"), qui renvoie la valeur unique quand une seule est sélectionnée, et la valeur de repli (BLANK par défaut) sinon. C'est plus sûr et plus lisible que de détourner MIN ou MAX, ou que d'écrire

à la main HASONEVALUE combiné à VALUES.

■ ■ ■ Contexte de filtre : là où les juniors décrochent

15. Quelle est la différence entre contexte de ligne et contexte de filtre ?

Réponse faible du junior : « C'est la même chose, le contexte c'est ce qui est filtré dans le rapport. »

Réponse attendue : Le contexte de ligne est la « ligne courante » : il existe dans une colonne calculée et à l'intérieur d'un itérateur (SUMX, FILTER), et il permet de référencer les valeurs de cette ligne. Le contexte de filtre est l'ensemble des filtres actifs venant des visuels, slicers, lignes et colonnes d'une matrice, et de CALCULATE : il restreint les données vues par une mesure. Une mesure répond au contexte de filtre ; le contexte de ligne, lui, n'agrège rien tout seul. Confondre les deux est la racine de la plupart des erreurs DAX.

16. À quoi sert vraiment CALCULATE ?

Réponse faible du junior : « CALCULATE, ça fait un calcul, comme son nom l'indique. »

Réponse attendue : CALCULATE évalue une expression en modifiant le contexte de filtre : il ajoute, remplace ou retire des filtres. C'est la seule fonction qui manipule directement le contexte de filtre, et c'est le coeur du DAX avancé. Par exemple `CALCULATE([CA], Produit[Categorie] = "Vélo")` force le filtre catégorie, quel que soit le visuel. Ce prédicat de colonne est un raccourci pour `FILTER(ALL(Produit[Categorie]), Produit[Categorie] = "Vélo")` : le ALL retire d'abord le filtre existant sur cette colonne, ce qui permet au prédicat de le remplacer au lieu de l'intersecter avec l'existant. C'est exactement ce que `KEEPFILTERS` (piège 19) vient annuler.

17. Pourquoi une mesure se comporte-t-elle différemment dans une colonne calculée ?

Réponse faible du junior : « Je mets ma mesure dans une colonne calculée et le résultat est faux, ou identique sur toutes les lignes. »

Réponse attendue : Quand tu références une mesure (ou que tu enveloppes une expression dans CALCULATE) à l'intérieur d'un contexte de ligne, il se produit une transition de contexte : le contexte de ligne courant est transformé en contexte de filtre. Sans cela, une expression dans une colonne calculée ne « voit » pas la ligne pour agréger. C'est aussi pour cela qu'une mesure référencée par son nom est implicitement entourée d'un CALCULATE. Comprendre la transition de contexte est un classique éliminatoire.

18. Comment retires-tu des filtres : ALL, REMOVEFILTERS ou ALLEXCEPT ?

Réponse faible du junior : « Je connais ALL, je le mets un peu partout. »

Réponse attendue : ALL(Table) ou ALL(Colonne) retire les filtres pour calculer une référence (par exemple un dénominateur « toutes catégories »). REMOVEFILTERS fait la même chose avec un nom plus explicite. ALLEXCEPT(Table, Col1, Col2) retire tous les filtres de la table sauf ceux des colonnes listées, utile pour un calcul « par client, toutes périodes confondues ». Le piège : ALL employé comme valeur, et non comme modificateur de CALCULATE, renvoie une table et pas un total ; on l'utilise dans CALCULATE ou dans un itérateur.

19. À quoi sert KEEPFILTERS ?

Réponse faible du junior : « Jamais utilisé, j'ajoute juste mes filtres dans CALCULATE. »

Réponse attendue : Par défaut, un filtre passé à CALCULATE remplace le filtre existant sur la même colonne. KEEPFILTERS conserve le contexte de filtre courant et l'intersecte avec ton filtre au lieu de l'écraser. Exemple : dans un visuel déjà filtré sur plusieurs catégories, CALCULATE([CA], KEEPFILTERS(Produit[Categorie] = "Vélo")) ne donne un résultat que si « Vélo » fait partie de la sélection en cours, au lieu de forcer la catégorie en ignorant le visuel.

20. Pourquoi le total de ta mesure ne correspond pas à la somme des lignes ?

Réponse faible du junior : « Le visuel est buggé, le total ne tombe pas juste. »

Réponse attendue : Le total n'est pas la somme des lignes : c'est la mesure réévaluée dans le contexte de filtre du total. Pour une mesure non additive (un ratio, une marge en pourcentage, un DISTINCTCOUNT), recalculer au niveau total donne logiquement un résultat différent de l'addition des lignes, et c'est correct. Le vrai sujet n'est pas un bug du visuel, c'est le contexte de filtre. Si tu veux que le total agrège les lignes, tu dois l'exprimer explicitement, par exemple avec un SUMX sur la granularité voulue.

■ ■ ■ Time intelligence : le problème est presque toujours la table de dates

21. Comment calcules-tu un stock ou un solde de fin de période ?

Réponse faible du junior : « Je fais un SUM, comme pour le chiffre d'affaires. »

Réponse attendue : Un stock, un solde ou un effectif est une mesure semi-additive : elle s'additionne entre produits ou magasins, mais pas dans le temps. Sommer le stock de chaque jour du mois n'a aucun sens. Tu prends la dernière valeur connue de la période, avec CLOSINGBALANCEMONTH, ou LASTNONBLANKVALUE qui ignore les jours sans mouvement, en t'appuyant sur la table de dates. Confondre additif et semi-additif donne des totaux gonflés que personne ne sait expliquer, et c'est un classique d'entretien sur les données de stock ou de RH.

22. Comment calcules-tu un cumul annuel (YTD)?

Réponse faible du junior : « `TOTALYTD([CA], Dates[Date])` et c'est réglé pour toute l'entreprise. »

Réponse attendue : `TOTALYTD` (ou `DATESYTD`) calcule le cumul depuis le 1er janvier par défaut. Si l'année fiscale ne se termine pas le 31 décembre, tu passes la date de fin d'exercice en dernier argument, par exemple pour une clôture au 30 juin. Sans ce paramètre, le cumul reste juste sur l'année civile mais devient faux sur l'exercice : une erreur silencieuse que beaucoup de juniors ne repèrent pas, parce que la mesure ne renvoie aucune erreur.

23. Comment compares-tu une période à l'an dernier?

Réponse faible du junior : « Je filtre sur `YEAR(date)` moins 1 à la main dans `CALCULATE`. »

Réponse attendue : Tu utilises les fonctions de time intelligence sur la table de dates : `SAMEPERIODLASTYEAR(Dates[Date])` pour le même intervalle décalé d'un an, ou `DATEADD(Dates[Date], -1, YEAR)`, plus général (jour, mois, trimestre, année). Elles s'appuient sur la continuité de la table de dates. Un `YEAR` moins 1 bricolé à la main casse dès qu'on filtre un trimestre, un mois glissant, ou qu'il manque des dates, et il gère mal les fins de mois.

24. `SAMEPERIODLASTYEAR` et `DATEADD` sont-ils interchangeables?

Réponse faible du junior : « C'est pareil, je prends le premier qui marche. »

Réponse attendue : `SAMEPERIODLASTYEAR` est un cas particulier : un décalage d'exactly un an. `DATEADD` est paramétrable : nombre de périodes et unité (`DAY`, `MONTH`, `QUARTER`, `YEAR`), ce qui permet un mois précédent, un trimestre précédent, etc. Les deux exigent une table de dates continue et marquée. Attention au piège des dates futures : si ta table de dates va jusqu'au 31 décembre mais que les ventes s'arrêtent en juin, un décalage peut comparer une vraie période à une période sans données, et afficher des écarts trompeurs.

25. Comment fais-tu un cumul glissant (running total) propre?

Réponse faible du junior : « Je crée une colonne calculée qui additionne les lignes précédentes. »

Réponse attendue : Un cumul se fait dans une mesure qui élargit le contexte de filtre sur les dates jusqu'à la date courante, typiquement `CALCULATE([CA], DATESYTD(Dates[Date]))` pour un cumul annuel, ou `CALCULATE([CA], FILTER(ALL(Dates), Dates[Date] <= MAX(Dates[Date])))` pour un cumul total. Une colonne calculée fige le résultat à l'actualisation, ignore les filtres du visuel et ne réagit pas aux slicers : c'est la mauvaise réponse dès qu'on veut un cumul interactif.

■ ■ ■ Performance & restitution : un modèle propre, pas un visuel rafistolé

26. Import ou DirectQuery : lequel choisis-tu et pourquoi ?

Réponse faible du junior : « DirectQuery, comme ça c'est toujours en temps réel. »

Réponse attendue : En mode Import, les données sont chargées et compressées dans le moteur VertiPaq en mémoire : c'est de loin le plus rapide et le plus riche en DAX, au prix d'une actualisation planifiée. DirectQuery interroge la source à chaque visuel : utile pour de très gros volumes ou un besoin de fraîcheur, mais plus lent et limité sur certaines fonctions DAX. Tu choisis Import par défaut, et DirectQuery seulement quand le volume ou la fraîcheur l'imposent, éventuellement en mode composite avec des tables d'agrégation pour accélérer les requêtes.

27. Pourquoi ton fichier est-il si lourd et si lent ?

Réponse faible du junior : « C'est le volume de lignes, on n'y peut rien. »

Réponse attendue : VertiPaq compresse colonne par colonne, et le facteur dominant n'est pas le nombre de lignes mais la cardinalité des colonnes (le nombre de valeurs distinctes). Une colonne à très haute cardinalité (identifiants uniques, horodatages à la seconde, texte libre) compresse mal et alourdit tout. Les leviers : supprimer les colonnes inutiles, réduire la cardinalité (séparer date et heure, arrondir, garder des clés entières) et limiter le grain. Souvent, retirer trois colonnes inutilisées fait plus que n'importe quelle optimisation DAX.

28. Tu as besoin d'une nouvelle colonne. Où la crées-tu ?

Réponse faible du junior : « Une colonne calculée en DAX, c'est rapide à faire. »

Réponse attendue : Une colonne calculée en DAX est recalculée à chaque actualisation, stockée, et souvent moins bien compressée qu'une colonne importée. Quand c'est possible, crée la colonne le plus en amont : à la source (SQL, entrepôt) ou dans Power Query. Tu réserves la colonne calculée DAX aux cas qui ont besoin du contexte du modèle (relations, autres tables) et qui ne peuvent pas être faits en amont. Et pour de l'agrégation, ce n'est pas une colonne, c'est une mesure.

29. L'option « Date/heure automatique » est cochée par défaut. Tu la laisses ?

Réponse faible du junior : « Je n'y touche pas, ça gère les hiérarchies de dates tout seul. »

Réponse attendue : La date/heure automatique crée une table de dates cachée pour chaque colonne de type date du modèle. Sur un modèle réel, cela multiplie les tables masquées, gonfle la taille du fichier et donne des hiérarchies de dates incohérentes d'une colonne à l'autre. La bonne pratique : désactiver l'option et brancher ta propre table de dates dédiée, unique, marquée comme table de dates, que tu contrôles.

30. Tu glisses un champ numérique dans un visuel et Power BI affiche la somme. C'est suffisant ?

Réponse faible du junior : « Oui, le glisser-déposer agrège tout seul, je n'écris pas de mesure. »

Réponse attendue : Le glisser-déposer crée une mesure implicite : tu perds le contrôle de l'agrégation, du format et du nom, et tu ne peux ni la réutiliser ni la composer dans d'autres calculs. La pratique attendue : écrire des mesures explicites, nommées, avec un format défini (devise, pourcentage, séparateurs de milliers) et une logique maîtrisée. Un modèle propre s'appuie sur des mesures explicites, pas sur des agrégations implicites improvisées au fil des visuels.